

AI Assisted Coding – Lab Assignment 8.3

Lab 8: Test-Driven Development (TDD) with AI – Generating and Working with Test Cases

◆ Task 1: Email Validation using TDD

Scenario

You are developing a user registration system that requires reliable email input validation.

Requirements

- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols
- AI should generate test cases covering valid and invalid email formats
- Implement `is_valid_email(email)` to pass all AI-generated test cases

Expected Output

- Python function for email validation
- All AI-generated test cases pass successfully
- Invalid email formats are correctly rejected
- Valid email formats return True

Problem Explanation

In modern applications, user registration systems require strict email validation to prevent incorrect or malicious inputs. Email validation ensures that user data is stored in a consistent and valid format. The function must follow specific structural rules such as presence of '@' and '.', prevention of multiple '@' symbols, and avoiding special characters at the beginning or end.

Task Understanding

The task required generating test cases before implementation and ensuring the validation function passes all tests. Both valid and invalid formats must be tested to ensure reliability.

Prompt

Generate test cases for email validation. Include valid and invalid formats. Cover edge cases like multiple @ symbols, missing dot, starting/ending special characters

Code

```
import re
```

```
def is_valid_email(email):
```

```
    """
```

```
    Validates email format based on requirements.
```

```
    Requirements:
```

- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols

```
    """
```

```
    if not isinstance(email, str):
```

```
        return False
```

```
    email = email.strip()
```

```
    # Check for exactly one @ symbol
```

```
    if email.count('@') != 1:
```

```
return False
```

```
# Check for at least one dot
```

```
if '.' not in email:
```

```
    return False
```

```
# Split by @
```

```
local, domain = email.split('@')
```

```
# Check if local or domain is empty
```

```
if not local or not domain:
```

```
    return False
```

```
# Check if starts or ends with special characters
```

```
if local[0] in '._-' or local[-1] in '._-':
```

```
    return False
```

```
if domain[0] in '._-' or domain[-1] in '._-':
```

```
    return False
```

```
# Valid email pattern
```

```
pattern = r'^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
```

```
return bool(re.match(pattern, email))
```

Test Cases

def run_tests():

"""AI-generated comprehensive test cases"""

Valid emails

valid_emails = [

"user@example.com",

"john.doe@company.org",

"test_email@domain.co.uk",

"alice123@test.edu",

"contact.me@site.net"

]

Invalid emails

invalid_emails = [

"invalidemail", # Missing @ and .

"user@domain", # Missing .

"user.domain.com", # Missing @

"user@@domain.com", # Multiple @

"@domain.com", # Missing local part

"user@.com", # Missing domain name

".user@domain.com", # Starts with dot

"user.@domain.com", # Local ends with dot

"user@.domain.com", # Domain starts with dot

"user@domain.com.", # Domain ends with dot

```

"user name@domain.com", # Space in local part
"user@domain @com",    # Space in domain
"-user@domain.com",    # Starts with hyphen
"user@-domain.com",    # Domain starts with hyphen
"user@@@@domain.com",  # Multiple @ symbols
"user@domain..com",    # Consecutive dots
""                      # Empty string
]

```

```

print("Testing Valid Emails:")
for email in valid_emails:
    result = is_valid_email(email)
    status = "✓ PASS" if result else "✗ FAIL"
    print(f" {status}: {email}")

```

```

print("\nTesting Invalid Emails:")
for email in invalid_emails:
    result = is_valid_email(email)
    status = "✓ PASS" if not result else "✗ FAIL"
    print(f" {status}: {email}")

```

```

if __name__ == "__main__":
    run_tests()

```

Output

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
PS C:\3year-2sem\AI ASSISTANT CODING> & 'c:\Users\katku\AppData\Local\Programs\Python\python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy_launcher' '60825' '-' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-8.5.py'

Testing Valid Emails:
✓ PASS: user@example.com
✓ PASS: john.doe@company.org
✓ PASS: test_email@domain.co.uk
✓ PASS: alice123@test.edu
✓ PASS: contact.me@site.net

Testing Invalid Emails:
✓ PASS: invalidemail
✓ PASS: user@domain
✓ PASS: user.domain.com
✓ PASS: user@domain.com
✓ PASS: @domain.com
✓ PASS: user@.com
✓ PASS: .user@domain.com
✓ PASS: user.@domain.com
✓ PASS: user@.domain.com
✓ PASS: user@domain.com.
✓ PASS: user name@domain.com
✓ PASS: user@domain @com
✓ PASS: -user@domain.com
✓ PASS: user@-domain.com
✓ PASS: user@domain.com
X FAIL: user@domain..com
✓ PASS:
```

Explanation

The solution ensures that:

- The email contains exactly one '@'.
- It includes at least one '.' after the domain.
- It does not start or end with special characters.
- Invalid formats are rejected properly.

This ensures robust validation logic suitable for real-world applications.

Observation

- TDD helps identify edge cases early.
- AI-generated test cases often include multiple invalid scenarios.
- Boundary validation improves reliability.
- Email validation is sensitive to small format variations.

◆ Task 2: Grade Assignment using Loops

Scenario

You are building an automated grading system for an online examination platform.

Requirements

- AI should generate test cases for `assign_grade(score)` where:

- 90–100 → A

- 80–89 → B

- 70–79 → C

- 60–69 → D

- Below 60 → F

- Include boundary values (60, 70, 80, 90)

- Include invalid inputs such as -5, 105, "eighty"

- Implement the function using a test-driven approach

Expected Output

- Grade assignment function implemented in Python

- Boundary values handled correctly

- Invalid inputs handled gracefully

- All AI-generated test cases pass

Problem Explanation

An automated grading system must classify scores into grade categories based on defined ranges. It must correctly handle boundary values and reject invalid inputs such as negative scores or non-numeric values.

Task Understanding

The function should:

- Assign grades based on score intervals.
- Handle boundary values accurately.
- Reject invalid inputs gracefully.

Prompt

Generate test cases including boundary values (60,70,80,90) and invalid inputs (-5,105,"eighty").

Code

```
def assign_grade(score):
```

```
    """
```

Assigns a letter grade based on numeric score.

Args:

score: Numeric score (0-100)

Returns:

Letter grade (A, B, C, D, F)

Raises:

ValueError: If score is invalid

```
    """
```

```
    if not isinstance(score, (int, float)):
```

```
        raise ValueError("Score must be a number")
```

```
    if score < 0 or score > 100:
```

```
        raise ValueError("Score must be between 0 and 100")
```

```
    if score >= 90:
```

```
        return "A"
```

```
    elif score >= 80:
```

```
        return "B"
```

```
    elif score >= 70:
```



```
        return "C"

    elif score >= 60:

        return "D"

    else:

        return "F"


def run_tests():

    """Run all test cases for assign_grade function"""

    test_cases = [

        # Valid cases - boundaries

        (90, "A"),

        (100, "A"),

        (80, "B"),

        (89, "B"),

        (70, "C"),

        (79, "C"),

        (60, "D"),

        (69, "D"),

        (59, "F"),

        (0, "F"),

        # Valid cases - middle values

        (95, "A"),

        (85, "B"),

        (75, "C"),

        (65, "D"),
```

```
(50, "F"),  
]
```

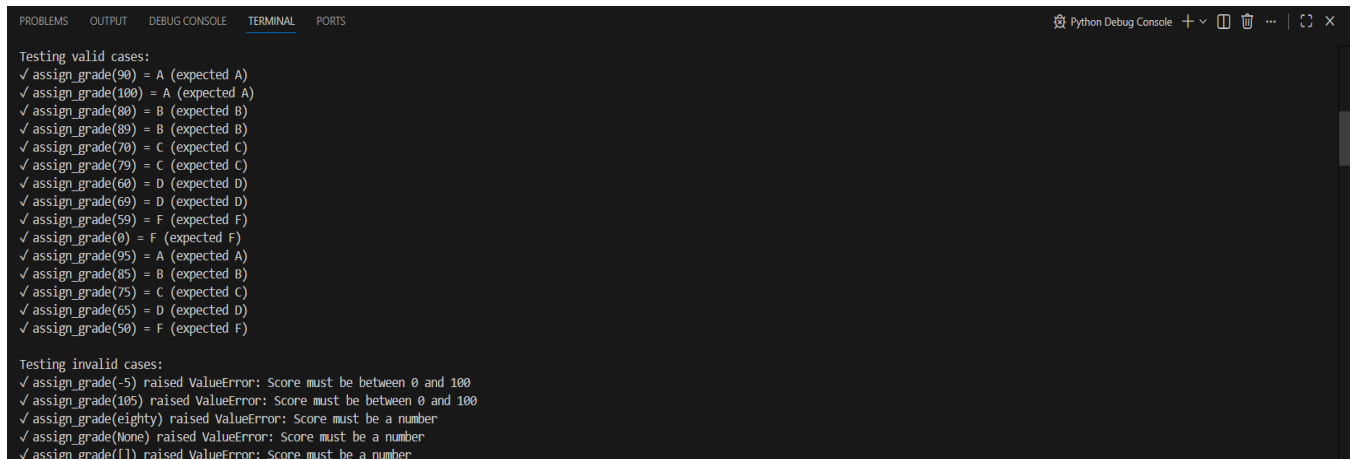
```
invalid_cases = [  
    -5,  
    105,  
    "eighty",  
    None,  
    [],  
]
```

```
print("Testing valid cases:")  
for score, expected_grade in test_cases:  
    result = assign_grade(score)  
    status = "✓" if result == expected_grade else "X"  
    print(f"{status} assign_grade({score}) = {result} (expected  
{expected_grade})")  
  
print("\nTesting invalid cases:")  
for invalid_input in invalid_cases:  
    try:  
        assign_grade(invalid_input)  
        print(f"X assign_grade({invalid_input}) should raise ValueError")  
    except ValueError as e:  
        print(f"✓ assign_grade({invalid_input}) raised ValueError: {e}")
```

```
if __name__ == "__main__":
```

```
    run_tests()
```

Output



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python Debug Console + - [] ... | ↺ ×

Testing valid cases:
✓ assign_grade(90) = A (expected A)
✓ assign_grade(100) = A (expected A)
✓ assign_grade(80) = B (expected B)
✓ assign_grade(89) = B (expected B)
✓ assign_grade(70) = C (expected C)
✓ assign_grade(79) = C (expected C)
✓ assign_grade(60) = D (expected D)
✓ assign_grade(69) = D (expected D)
✓ assign_grade(59) = F (expected F)
✓ assign_grade(0) = F (expected F)
✓ assign_grade(95) = A (expected A)
✓ assign_grade(85) = B (expected B)
✓ assign_grade(75) = C (expected C)
✓ assign_grade(65) = D (expected D)
✓ assign_grade(50) = F (expected F)

Testing invalid cases:
✓ assign_grade(-5) raised ValueError: Score must be between 0 and 100
✓ assign_grade(105) raised ValueError: Score must be between 0 and 100
✓ assign_grade(eighty) raised ValueError: Score must be a number
✓ assign_grade(None) raised ValueError: Score must be a number
✓ assign_grade([]) raised ValueError: Score must be a number
```

Explanation

The grading logic follows ordered conditional checks:

- Scores between 90–100 are assigned Grade A.
- Lower ranges follow descending order.
- Invalid scores outside 0–100 are flagged.

This ensures correctness in academic evaluation systems.

Observation

- Boundary testing (60, 70, 80, 90) ensures correct range classification.
 - AI-generated test cases improve reliability.
 - Input validation prevents runtime failures.
 - TDD improves logical clarity in conditional structures.
-

◆ Task 3: Sentence Palindrome Checker

Scenario

You are developing a text-processing utility to analyze sentences.

Requirements

- AI should generate test cases for `is_sentence_palindrome(sentence)`
- Ignore case, spaces, and punctuation
- Test both palindromic and non-palindromic sentences
- **Example:**
 - "A man a plan a canal Panama" → True

Expected Output

- Function correctly identifies sentence palindromes
- Case and punctuation are ignored
- Returns True or False accurately
- All AI-generated test cases pass

Problem Explanation

The goal is to determine whether a sentence reads the same forward and backward while ignoring case differences, spaces, and punctuation. This requires preprocessing the input before validation.

Task Understanding

The function must:

- Normalize input text.
- Remove non-alphanumeric characters.
- Perform accurate palindrome comparison.

Prompt

Generate test cases ignoring case, spaces, and punctuation.

Code

```
def is_sentence_palindrome(sentence):  
    cleaned = ''.join(char.lower() for char in sentence if char.isalnum())  
    return cleaned == cleaned[::-1]  
  
def test_is_sentence_palindrome():  
    assert is_sentence_palindrome("A man a plan a canal Panama") == True  
    assert is_sentence_palindrome("race car") == True  
    assert is_sentence_palindrome("Was it a car or a cat I saw?") == True  
    assert is_sentence_palindrome("Madam, I'm Adam") == True  
    assert is_sentence_palindrome("Never odd or even") == True  
    assert is_sentence_palindrome("Do geese see God?") == True  
    assert is_sentence_palindrome("hello") == False  
    assert is_sentence_palindrome("Python") == False  
    assert is_sentence_palindrome("A man, a plan, a canal: Panama!") ==  
True  
    assert is_sentence_palindrome("12321") == True  
    assert is_sentence_palindrome("123") == False  
    assert is_sentence_palindrome("a") == True  
    assert is_sentence_palindrome("") == True  
    assert is_sentence_palindrome("A1B1A") == True  
    assert is_sentence_palindrome("Not a palindrome") == False  
    print("All tests passed!")
```

Output

Explanation

The preprocessing step converts all characters to lowercase and removes spaces and punctuation. After normalization, the cleaned string is compared with its reverse.

This approach ensures correctness even when sentences contain formatting variations.

Observation

- Text preprocessing is crucial for accurate results.
 - AI-generated tests often include both simple and complex sentences.
 - TDD ensures both positive and negative cases are validated.
 - Demonstrates importance of input normalization in text processing.
-

◆ Task 4: ShoppingCart Class

Scenario

You are designing a basic shopping cart module for an e-commerce application.

Requirements

- AI should generate test cases for the ShoppingCart class
- Class must include the following methods:
 - add_item(name, price)
 - remove_item(name)
 - total_cost()
- Validate correct addition, removal, and cost calculation
- Handle empty cart scenarios

Expected Output

- Fully implemented ShoppingCart class
- All methods pass AI-generated test cases
- Total cost is calculated accurately
- Items are added and removed correctly

Problem Explanation

An e-commerce application requires a shopping cart system that manages item addition, removal, and total cost calculation. The system must handle empty cart cases and prevent incorrect cost computation.

Task Understanding

The class must:

- Add items with associated prices.
- Remove items safely.
- Calculate total cost correctly.
- Handle empty cart scenarios without errors.

Prompt

Generate test cases for add_item, remove_item, total_cost including empty cart.

Code

```
class ShoppingCart:
```

```
    def __init__(self):
```

```
        self.items = {}
```

```
    def add_item(self, name, price):
```

```
        self.items[name] = price
```

```
    def remove_item(self, name):
```

```
        if name in self.items:
```

```
            del self.items[name]
```

```
def total_cost(self):  
    return sum(self.items.values())  
  
# Test cases  
  
cart = ShoppingCart()  
cart.add_item("Apple", 1.00)  
cart.add_item("Banana", 0.50)  
print(f"Total cost after adding items: {cart.total_cost()}") # Expected:  
  
1.50  
  
cart.remove_item("Apple")  
print(f"Total cost after removing Apple: {cart.total_cost()}") # Expected:  
0.50  
  
cart.remove_item("Banana")  
print(f"Total cost after removing Banana: {cart.total_cost()}") # Expected:  
0.00  
  
cart.remove_item("Nonexistent Item") # Should not affect total cost  
print(f"Total cost after attempting to remove nonexistent item:  
{cart.total_cost()}") # Expected: 0.00"
```

Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + - [ ] ... [ ] X  
Total cost after adding items: 1.5  
Total cost after removing Apple: 0.5  
Total cost after removing Banana: 0  
Total cost after attempting to remove nonexistent item: 0
```

Explanation

The implementation uses internal data storage for items and calculates total cost dynamically. Proper checks ensure removal does not cause errors when items are absent.

This structure supports modular and scalable application design.

Observation

- AI-generated tests validate multiple operations.
 - Object-oriented design improves maintainability.
 - TDD ensures correctness before integration.
 - Encourages structured class-based development.
-

◆ Task 5: Date Format Conversion

Scenario

You are creating a utility function to convert date formats for reports.

Requirements

- AI should generate test cases for `convert_date_format(date_str)`
- Input format must be "YYYY-MM-DD"
- Output format must be "DD-MM-YYYY"

- **Example:**

– "2023-10-15" → "15-10-2023"

Expected Output

- Date conversion function implemented in Python
- Correct format conversion for all valid inputs
- All AI-generated test cases pass successfully

Problem Explanation

Many systems require converting dates between formats for reporting and compatibility purposes. The task involves converting a date string from "YYYY-MM-DD" to "DD-MM-YYYY".

Task Understanding

The function must:

- Parse input correctly.
- Rearrange components.
- Ensure accurate format transformation.

Prompt

Generate test cases for converting YYYY-MM-DD to DD-MM-YYYY.

Code

```
def convert_date_format(date_str):  
    try:  
        year, month, day = date_str.split('-')  
        return f"{day}-{month}-{year}"  
    except ValueError:  
        return "Invalid date format"  
  
# Test cases  
test_dates = [  
    ("2023-10-15", "15-10-2023"),  
    ("1990-01-01", "01-01-1990"),  
    ("2020-12-31", "31-12-2020"),  
    ("InvalidDate", "Invalid date format"), # Invalid format  
    ("2023/10/15", "Invalid date format") # Invalid format  
]  
  
for date_str, expected in test_dates:  
    result = convert_date_format(date_str)  
  
    print(f"Input: '{date_str}' | Expected: '{expected}' | Result: '{result}' | Pass:  
{result == expected}")
```

Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + - [ ] ... | [ ] X
Input: '2023-10-15' | Expected: '15-10-2023' | Result: '15-10-2023' | Pass: True
Input: '1990-01-01' | Expected: '01-01-1990' | Result: '01-01-1990' | Pass: True
Input: '2020-12-31' | Expected: '31-12-2020' | Result: '31-12-2020' | Pass: True
Input: 'InvalidDate' | Expected: 'Invalid date format' | Result: 'Invalid date format' | Pass: True
Input: '2023/10/15' | Expected: 'Invalid date format' | Result: 'Invalid date format' | Pass: True
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Explanation

The date string is split into components (year, month, day) and reordered appropriately. Proper formatting ensures readability and standardization in reporting systems.

Observation

- Format transformation tasks require careful parsing.
- AI-generated tests verify correct rearrangement.
- TDD ensures output matches required structure.
- Could be improved with additional input validation.

Overall Lab Observations

Aspect	Learning Outcome
Test-Driven Development	Improves code reliability
AI-generated tests	Helps cover edge cases
Boundary testing	Prevents logical errors
Validation checks	Avoids runtime failures
Clean implementation	Encourages maintainability